LEDGER · PROJECT HANDBOOK

Your accounting project, explained.

What each screen does.

Where every action stores data.

How the accounting records connect.

> Includes sales, purchases, expenses, users and roles, banking, reports, subscriptions and a complete 51-table database reference.

Prepared 7 October 2026 · Asia/Karachi

Based on the source code and migrated local schema.

This guide explains the implemented Ledger application. Examples are teaching examples, not transactions inserted into your database. No login passwords, credentials or private customer records are included.

READING THIS GUIDE

Contents and key terms

Section	What you will learn
1	Project architecture and the path from a screen to the database.
2	Quick action-to-table map for the whole application.
3	Company creation, switching and tenant isolation.
4	User registration, authentication, memberships, roles and permissions.
5	Customers, suppliers, contact imports and histories.
6	Sales invoices, approval, customer receipts and email/PDF.
7	Purchases: bill creation, approval, supplier payments and voiding.
8	Direct paid expenses and nested expense categories.
9	Credit notes and customer refunds.
10	Chart of accounts, journals, opening balances and reversals.
11	Bank accounts, CSV statements, matching and reconciliation sessions.
12	Dashboard and every implemented financial report.
13	Attachments, report exports, queues and audit logs.
14	Company settings, taxes, currencies and financial-year locks.
15	Subscriptions, platform plans, billing and verified payments.
16	Worked purchase/sale examples, debugging and development setup.
17	Implementation boundaries and source-file navigation.
Appendix A	Every current table, column, key and foreign-key relationship.

Table: a named collection of database records. **Row:** one record. **id:** its identifier. **Foreign key:** a reference to a related row. **company_id:** the company that owns a tenant record. **Header:** the overall document. **Items:** its individual lines. **Journal:** the official double-entry accounting record.

Insert creates a row; **update** changes a row; **read** displays/calculates existing information. Most reports read records without creating new financial transactions.

From a button to a database row

Vue page → Inertia request → Laravel route → validation and authorization → controller → business service → Eloquent/database transaction → response

Laravel 12 handles the backend. Vue 3 with TypeScript provides the interactive screens. Inertia connects those screens to Laravel controllers, so application navigation uses Laravel routes rather than a separate Vue Router. Tailwind styles the interface and Vite builds the frontend.

The current local app uses `database/database.sqlite`. PostgreSQL is supported for later deployment. Laravel Eloquent models represent tables; Form Requests validate input; policies and permission checks protect operations. Dedicated services centralize accounting, banking, memberships and subscription behavior.

Folder	Purpose
`resources/js/Pages`	Dashboard and business screens: Documents, Contacts, Purchases, Accounting, Banking, Reports and Settings.
`resources/js/Components`	Reusable tables, form fields, money displays, user forms and permission editor.
`routes`	Page/action URLs, authentication and the scheduler.
`app/Http`	Controllers, validation requests and company middleware.
`app/Services`	Transactions and business rules.
`app/Models`	Table relationships, tenant scopes and immutability checks.
`database`	Migrations define tables; seeders insert initial reference/demo data.
`app/Jobs`	Invoice email and generated report jobs.
`tests/Feature`	Automated workflow, access and accounting checks.

Two kinds of business records

Invoices and bills describe business documents. Journal entries and journal lines describe their accounting effect. Saving a draft document does not affect profit, payable or receivable ledger balances. Approval creates its posted accounting record. Paying a posted document creates a separate settlement journal.

A database transaction groups related changes together. If posting fails because an entry is unbalanced or the period is locked, its linked financial changes roll back together.

02 · QUICK STORAGE MAP

Where each action stores data

“Plus audit” below means an event is inserted into `audit_logs`. Journal posting itself also records an audit event.

Action	Main writes
Register or create a user	`users`; direct company user creation also inserts `company_users` plus audit.
Create a company	`companies`, `company_users`, default `accounts`, `bank_accounts`, `expense_categories`, `tax_rates`, `document_sequences`, `financial_years`; initial `subscriptions` when a plan exists.
Assign a user's role or activation	Update `company_users.role_id/is_active`, plus audit.
Customize role permissions	Replace that company's role rows in `company_permission_overrides`, plus audit.
Invite / accept a user	`company_invitations`; acceptance inserts `company_users` and updates `accepted_at`, plus audit.
Create / import a contact	`customers` or `suppliers`, plus audit.
Create a sales draft	`invoices`, `invoice_items`; increment `document_sequences`, plus audit.
Approve a sales invoice	`journal_entries`, `journal_lines`; update `invoices`, plus audit.
Receive customer payment	`payments`, journals; update invoice payment summary/status, plus audit.
Create a purchase draft	`bills`, `bill_items`; increment numbering sequence, plus audit.
Approve a purchase bill	Journals; update `bills.status/journal_entry_id`, plus audit.
Pay a supplier	`bill_payments`, journals; update bill payment summary/status, plus audit.
Record a paid expense	`expenses` and journals, plus audit.
Issue a credit note	`credit_notes`, `credit_note_items`, journals; update invoice credit total/status and sequence, plus audit.
Refund a customer	`customer_refunds`, journals; reduce invoice `paid_total`, plus audit.
Void or reverse a posting	New opposite journals; original journals retained. Voiding updates the original invoice/bill status, plus audit.

Storage map, continued

Action	Main writes or reads
Create / edit an account	`accounts`, plus audit. It does not create a financial balance on its own.
Create a bank account	`bank_accounts` references an existing asset account, plus audit.
Import a bank statement	`bank_transactions`, plus audit. It does not automatically post journals.
Match a bank transaction	`reconciliations`, plus audit. Existing journals are linked, not recreated.
Open / close statement session	`reconciliation_sessions`, plus audit; closing reads bank postings and matches.
Upload a receipt or document	`attachments` stores metadata; the actual file goes to private storage, plus audit.
Read dashboard / reports	Reads journals, accounts and relevant business records; no new financial rows.
Generate a large ledger export	`report_exports`, local `jobs`; generated PDF/XLSX is stored privately.
Email an invoice	Local `jobs`, cache lock; update `invoices.status` to sent after successful mail processing, plus audit.
Update company profile / lock date	`companies`, plus audit.
Add tax rate / financial year	`tax_rates` / `financial_years`, plus audit.
Close a financial year	Update `financial_years.is_closed` and `companies.locked_through`, plus audit.
Create / edit a platform plan	`subscription_plans` contains plan price, features and availability.
Select / change / cancel plan	`subscriptions`; a paid plan change creates `subscription_invoices`, plus audit.
Renew / settle subscription	`subscription_invoices` / `subscription_payments`; update subscription/invoice statuses, plus audit.

> There is no generic `purchases` table. Supplier purchases use `bills` and `bill_items`. Direct already-paid expenses use `expenses`.

Company creation and isolation

Open the company selector to create or switch a company. `companies` holds its name, legal/contact details, currency, timezone, financial-year start, status and lock date. A user can belong to several companies through `company_users`.

What creating a company inserts

1. One company row and an active Owner membership for its creator.
2. Eight default chart accounts: operating bank, receivables, input tax, payables, output tax, owner equity, sales and expenses.
3. A bank-account record linked to the operating-bank asset account.
4. A general expense category and a zero-rate default tax definition.
5. Independent invoice, bill and credit numbering sequences.
6. A financial year beginning on the selected start date.
7. A 14-day trial subscription using the first active platform plan, when available.

What switching changes

Switching stores the selected `company_id` in the authenticated session. With the current database session driver, the session payload is stored in `sessions`. Switching does not copy transactions between companies. The user opens a feature allowed by their role.

How access is checked

The `SetCompany` middleware verifies active membership and an active company. `CompanyContext` supplies the current company. Tenant models automatically filter queries and populate `company_id` on creation; they fail closed without context. Composite foreign keys tie children to parents in the same company. Permission checks still apply after membership is established.

Global reference tables include users, roles, permissions, currencies, exchange rates and subscription plans. A global user is shared, but their company roles and company records remain separate. PostgreSQL row-level security is not currently enabled.

Code: app/Services/Tenancy/CreateCompany.php; CompanyContext.php; CompanyHome.php; app/Http/Middleware/SetCompany.php; app/Models/TenantModel.php.

Users, roles and permissions

Open **Settings → Users & roles** at `/settings/users`. An Owner or a user with `users.manage` can manage company access.

Record	What is stored
`users`	One login identity: name, unique email, password hash, email verification, remember token and platform-admin flag. Passwords are hidden from model JSON.
`company_users`	company_id + user_id + role_id + is_active. One user has one membership/role per company.
`roles` / `permissions`	Role names and permission names such as invoice.approve or banking.reconcile.
`role_permissions`	Default permission pairs for each shared role.
`company_permission_overrides`	Company + role + permission + allowed. Permission editing replaces the selected company's override set.

Creating a new account

Click **New user**, fill name, email, password/confirmation and role, then create. The backend validates a password of at least 12 characters with letters and numbers, hashes it, inserts the user and active membership, records an audit event and dispatches registration after commit. The user must verify their email. Existing emails cannot be overwritten; use an invitation for an existing account.

Role defaults

Role	Default purpose
Owner / Administrator	All seeded company permissions; Owner permissions cannot be reduced.
Accountant	Financial operations and reports, excluding company/user administration.
Sales Manager	Selected contact and invoice creation, editing, approval and send actions.
Employee	View contacts and expenses; create paid expenses.
Read-only User	Seeded view permissions; no default create/post/pay permissions.

Choose a member's role and Active checkbox, then **Save access**. Only that company's membership changes. Deactivation retains global identity and historical attribution. The last active Owner is protected. Creation/reactivation and outstanding invitations count toward plan limits.

Invitations and authentication

Inviting someone

An invitation creates `company_invitations` with email, role, expiry and a hashed token. The UI shows the original private invitation URL once through session flash. It must be shared with the recipient; automatic invitation-email delivery is not currently implemented.

The recipient registers or logs in, verifies their email, then accepts the link. Acceptance checks email, expiry and prior use, creates membership, sets `accepted_at` and records audit history. Revoking deletes the unused invitation and records an audit event. Duplicate pending invites and invites to existing members are rejected.

Login, verification and password reset

Login reads the stored password hash and writes session state. Logout invalidates the session. Email verification updates `users.email_verified_at`; it does not create a special verification table. Password reset uses hashed tokens in `password_reset_tokens` and updates the user's password hash. Profile editing writes the global user's name/email; changing email clears verification. The profile cannot be deleted while it still has company memberships.

Platform administration

The `users.is_platform_admin` flag controls platform plan management and verified subscription payments. It is separate from an Owner's company role. Granting it is a trusted-server operation through `php artisan ledger:admin EMAIL`, never the user-creation form.

Code: CompanyUserController; CreateCompanyUserRequest; MembershipService; CompanyPermissions; PermissionController; Settings/Users.vue; Breeze Auth controllers; PlatformAdministrator command.

Customers and suppliers

Sales customers are stored in `customers`; purchase suppliers are stored in `suppliers`. Both carry `company_id`, name, email, phone, address, tax number and active/archived status. They are independent contact collections, not parent/child tables.

Action	Storage and effect
Create / edit	Insert/update the appropriate contact row and audit event. No journal is created.
Archive	Set status to archived. Linked invoices/bills and historical rows remain.
CSV import	Validate every row first; insert new name/email pairs in one transaction. Existing pairs are skipped.
Contact history	Read invoices where customer_id matches, or bills where supplier_id matches.
Posted balance	Read linked posted control-account journal lines, including payments, credits/refunds and reversals.
Excel export	Read company contacts and stream an XLSX file; no financial rows are inserted.

The contact CSV header is `name,email,phone,address,tax_number`. The current limit is 10,000 rows / 5 MB. Document history is server paginated.

A contact row has no stored “balance” column. The current UI balance is calculated from accounting records. A supplier's open balance is normally payable; a customer's open balance is normally receivable. An overpaid/credited customer can have a negative receivable balance.

Code: ContactController; ContactImportController; BusinessExportController; OperationalReports::statement; resources/js/Pages/Contacts.

Sales invoices and receipts

1. Save an invoice draft

`invoices` stores the header: customer_id, invoice_number, issue_date, due_date, currency, status, subtotal, tax_total, total, paid_total, credited_total and notes. `invoice_items` stores one row per line with invoice_id, account_id, description, quantity, unit_price, fixed discount and tax snapshot. Numbering increments the company's invoice row in `document_sequences`. Status starts as draft; no journal is posted.

> **Calculation:** line subtotal = round(quantity × unit_price − fixed discount). Tax = round(subtotal × snapshotted rate / 100). Total = subtotal + tax. Invoice totals are sums of line totals. Quantity and price support four decimal places; posted amounts are rounded to two decimals with BCMath.

2. Approve and post

The invoice must be a positive draft. A journal is inserted and posted; its ID is written to `invoices.journal_entry_id`, and status becomes approved.

Approval account	Debit	Credit
Accounts receivable	Invoice total	—
Sales/revenue accounts	—	Before-tax line amounts
Output tax, when applicable	—	Tax total

3. Receive money

A customer receipt inserts `payments` with invoice_id, bank_account_id, amount, paid_at, reference, journal_entry_id, idempotency_key and request_hash. It creates a settlement journal: **debit Bank; credit Accounts receivable**. It increases invoice paid_total and sets partially_paid or paid. The amount cannot exceed the outstanding amount; the payment date cannot precede the invoice date.

> Current outstanding balance = total − paid_total − credited_total. It is a computed property, not a separate stored column. A paid status may reflect settlement by money and credits together.

Code: DocumentController; DocumentService::create/approve/pay; Invoice and Document models; Documents/Create.vue and Show.vue.

Invoice editing, sending and voiding

Editing is restricted to drafts. The service replaces draft items, recalculates totals and updates the header while keeping its document number. Posted financial fields and items are protected from ordinary mutation.

PDF and email

A PDF download reads the invoice/customer/items/company and renders a file. It does not add a financial transaction or automatically insert an attachment row. Email dispatch queues `SendInvoice` after commit. The worker checks company permission again, obtains a cache lock, renders a PDF and sends it through Laravel mail. Successful processing changes approved → sent and writes an invoice.sent audit event. Local mail is logged; a real SMTP transport is needed for delivery.

Voiding

Only approved/sent invoices with no payments and no credits can be voided. The service adds opposite journal lines in a new entry, sets its reversal_of_id to the original journal, changes invoice status to void and logs the action. The original journal remains posted; reports include both entries at their respective dates. The original invoice retains its original journal link.

Statuses and repeat requests

The implemented lifecycle uses draft, approved, sent, partially_paid, paid and void. Overdue is derived from due date and unpaid balance rather than a persisted scheduled status. Customer receipts use an idempotency key plus request hash: an exact retry returns the existing receipt; changing its payload while reusing the key is rejected.

There is no separate “invoice history” table. The header, items, payment records, credit records, journal links and audit events together form history. Initial invoice sending is implemented; later resend workflows are not.

Where a purchase is stored

Supplier → bills → bill_items → approval journal → bill_payments → bank matching

1. Create a supplier bill

Open **Purchases → Bills**. Select a supplier and enter the document dates and lines. `bills` stores one header with supplier_id, bill_number, bill_date, due_date, currency, notes, draft status and totals. `bill_items` stores each line's bill_id, expense/asset account, quantity, price, discount and tax. The bill numbering sequence advances and an audit event is inserted. Saving a draft does not yet affect accounting.

2. Approve the bill

The approval creates `journal_entries` and `journal_lines`. The bill gets approved status and journal_entry_id.

Approval account	Debit	Credit
Selected expense or asset accounts	Before-tax purchases	—
Input tax, when applicable	Tax total	—
Accounts payable	—	Bill total

Expense-account purchases affect profit; an asset-account purchase increases assets. The software records input tax when a configured rate applies. Tax treatment must be configured appropriately for your business; this guide documents code behavior.

3. Pay the supplier

Insert `bill_payments`, not customer `payments`. The row links bill_id, bank_account_id and journal_entry_id, with amount/date/reference and retry-protection fields. The settlement journal is **debit Accounts payable; credit Bank**. Update bills.paid_total and status. Partial payments keep an outstanding payable; full settlement reduces it to zero.

Code: DocumentService with kind bill; Bill/BillItem/BillPayment models; Documents pages; BusinessRecordsExport.

Purchase relationships and history

Relationship	Meaning
suppliers.id → bills.supplier_id	One supplier can have many purchase bills.
bills.id → bill_items.bill_id	One bill has one or more item rows.
accounts.id → bill_items.account_id	The line's purchase classification.
bills.journal_entry_id → journal_entries.id	The original approved purchase entry.
bills.id → bill_payments.bill_id	One bill can have several partial payments.
bill_payments.journal_entry_id → journal_entries.id	Each supplier payment has its own settlement entry.
bank_accounts.account_id → accounts.id	The bank metadata points to the ledger asset account used for the payment.

Example: an approved supplier bill is 1,000 before tax plus 100 tax = 1,100. Its purchase journal debits purchases 1,000 and input tax 100, credits payables 1,100. Paying 400 creates one bill_payments row, debits payables 400 and credits bank 400. Bill paid_total becomes 400 and outstanding becomes 700.

Draft editing replaces bill items and recalculates its totals. Ordinary edits to posted totals are blocked. Voiding an unpaid, uncredited approved bill creates an opposite journal and changes its status to void. Payment is recorded against one bill at a time; multi-bill supplier-payment batches and supplier credit-note workflows are not implemented.

> Do not also enter the same supplier bill as a direct expense. Bill approval already records the purchase; payment records settlement rather than a second purchase expense.

A bill PDF or Excel export reads existing bill data; it does not create another bill or journal. Bank statement imports are independent records that can later be matched against the payment's bank journal line.

Expenses and categories

Open **Purchases → Expenses** for a direct already-paid expense, such as office supplies purchased immediately from the bank. This workflow posts immediately; it is not a draft supplier bill.

Table	Stored information
`expense_categories`	Name, optional parent_id for nested categories and account_id for an expense ledger account.
`expenses`	Category, bank, date, description, before-tax amount, tax rate/tax amount, payment method, posted status, journal link and idempotency key/hash.
`journal_entries` / `journal_lines`	Debit expense account; debit input tax when applicable; credit bank for the gross amount.
`attachments`	Receipt-file metadata linked by model_type = expense and model_id = expense.id.
`audit_logs`	Expense posting and optional attachment activity.

> A direct expense of 100 plus tax of 20 posts: debit expense 100; debit input tax 20; credit bank 120. Profit reduces by the expense amount, while bank reduces by the gross payment.

The service requires an expense category tied to an expense account and a bank in the company's base currency. Inactive taxes are rejected. The frontend supplies a retry key; an identical retry returns the prior expense, while a changed payload with the same key is rejected.

Open a listed expense to view amounts, category, bank, private receipts and its journal if you have accounting permission. Posted expense records are immutable through ordinary model updates. Expense edit/reversal and optional approval stages are not implemented. Nested category creation exists as a protected backend endpoint, with no dedicated category management page yet.

Code: ExpenseController; ExpenseRequest; ExpenseService; ExpenseCategoryController; Purchases/Expenses.vue; ExpenseDetail.vue.

Credit notes and refunds

Issue a credit note

A posted invoice can receive credits against its original lines. The request selects invoice_item_id and a positive amount before tax. The credit cannot exceed that line's remaining uncredited subtotal. Tax is calculated from the original line's snapshot; the final credit absorbs remaining tax rounding.

`credit_notes` stores invoice_id, credit_number, date, gross amount, reason, status and journal link. `credit_note_items` stores each credited invoice_item_id, account_id, description, before-tax amount and tax_amount. The credit sequence advances. Posting updates invoices.credited_total and settlement status.

Credit-note posting	Debit	Credit
Original revenue accounts	Credited before-tax amounts	—
Output tax	Credited tax	—
Accounts receivable	—	Gross credit amount

Refund money to the customer

A refund requires an unrefunded posted credit and an actual customer credit balance. It inserts `customer_refunds` linked to the credit note, bank and settlement journal, with amount/date/key/hash. Posting is **debit Accounts receivable; credit Bank**. The service reduces the related invoice's paid_total, and records audit history.

> An invoice totals 120 and the customer pays 120. A credit for 24 makes its outstanding receivable −24. Refunding 24 reduces net paid_total from 120 to 96. Outstanding then becomes 120 − 96 − 24 = 0.

A credit date cannot precede its invoice; a refund date cannot precede the credit. Refund retries use idempotency checks. No independent draft approval screen for credit notes is implemented: the issue operation creates and posts the credit together.

Code: CreditService::issue/refund; CreditController; CreditNoteForm; Documents/Credits.vue.

Accounts, journals and reversals

`accounts` is the chart of accounts, not user login accounts. Its code/name/type/parent define classification; system_key identifies control accounts; cash_flow_class is operating, investing or financing. Ledger balances are calculated from journal lines, not stored in a balance column.

`journal_entries` stores date, reference, status, created_by, source_type/source_id, posted_at and optional reversal_of_id. `journal_lines` stores entry_id, account_id, debit, credit and description. All are company-owned.

Posting rules

1. Company membership and operation permission are enforced by protected routes.
2. Posting must be outside locked/closed accounting periods.
3. Each entry must contain at least two valid active-account lines from the same company.
4. Each line has exactly one positive debit or positive credit.
5. Total debits must equal total credits, using rounded decimal strings.
6. Insert header/lines, mark posted and record audit within a transaction.

Manual journals and opening balances

Manual journals use the same JournalService. The UI posts directly; a separate draft-review workflow is not enabled. Opening balances can be entered as a balanced manual journal, for example debit bank and credit opening equity.

Reversals

Reversing a manual journal creates another entry with debit/credit swapped. The new reversal_of_id references the original entry; its date cannot precede the original date. Each entry can be reversed once. Automatic entries must be adjusted through the originating document workflow, preserving document/payment summaries.

Posted journal headers and lines cannot be silently edited/deleted through the application. PostgreSQL adds immutable/balanced-posting triggers; SQLite local testing relies on the application checks plus available relational constraints. Account types with existing history cannot change; system controls must remain active with their original type.

Code: JournalService; AccountService; JournalEntry/JournalLine models; ImmutableRecord/ImmutableWhenPosted traits; AccountingController.

Bank statements and reconciliation

A bank account is stored in `bank_accounts` and points to an asset row in `accounts`. Bank balances are the posted debits minus credits for that ledger account. Creating bank metadata alone does not create an opening balance.

Importing CSV transactions

The CSV columns are `date,description,amount,reference`. Import creates `bank_transactions` containing the bank link, signed amount, transaction_type, reference, imported_at and import_hash. Positive amounts are receipts; negative amounts are payments. A fingerprint of date, amount, description and reference prevents duplicate rows for the same bank/company. Identical legitimate lines need distinct references. Zero-value lines are rejected.

> Importing a statement does not create journal entries. The statement records what the bank reports; posting records what the books contain. Reconciliation connects the two.

Matching an imported row

Select an existing posted bank journal line. The line must belong to the imported transaction's bank ledger account, and its signed debit-minus-credit amount must exactly equal the statement amount. Neither transaction nor line may already be matched.

A successful match inserts `reconciliations` with bank_transaction_id, journal_entry_id, journal_line_id, matched_amount, status and reconciled_at, plus an audit event. It does not recreate the payment. Match state is determined by the reconciliation row rather than updating a special status column in bank_transactions.

Statement sessions

`reconciliation_sessions` stores bank_account_id, statement_date, closing_balance, status and closed_at. Closing verifies that all eligible imported transactions through that date are matched and the posted bank balance agrees with the statement's closing amount. The session is marked closed and audited.

The current matching operation does not populate session_id on reconciliations. Sessions certify the bank's complete eligible history through the end date; independent session allocation, split/group matches, undo and live bank feeds are not implemented.

Code: ReconciliationService; SessionService; BankingController; BankAccountController; BankSetup.vue.

Dashboard and financial reports

Dashboard cards and charts read the active company's actual records. Revenue, expenses and profit are derived from posted journal lines; bank balances use bank-linked accounts. Outstanding invoice/bill summaries use document totals minus settlement/credits. Recent/overdue/upcoming lists read company documents. Reports do not need their own “report transactions” table.

Report	Main tables and calculation
Profit and Loss	accounts + posted journal_entries/journal_lines in range. Revenue is credits minus debits for revenue accounts; expenses are debits minus credits for expense accounts.
Balance Sheet	Posted account totals through the as-of date. Assets, liabilities and equity are grouped by account type; cumulative profit is shown as retained earnings.
Trial Balance	Group posted debits/credits by account for the selected range. Displayed balance is debit − credit; the current screen is period movement.
General Ledger	Posted journal lines with account/entry metadata; paginated and filterable.
Account Transactions	The same ledger query restricted by account_id.
Accounts Receivable	Posted invoices, customers, payments, credit notes/refunds and journal/reversal dates. Calculate as-of remaining balance and due-date age.
Accounts Payable	Posted bills, suppliers, bill_payments and journal/reversal dates; calculate as-of remaining payable.
Customer Statements	Invoice-linked receivable journal lines, receipts, credits, refunds and reversals; opening/running/closing balance.
Supplier Statements	Bill-linked payable journal lines, payments and reversals; opening/running/closing balance.
Expense Summary	Expense-account rows from the posted trial-balance calculation.
Tax Summary	Posted journal lines in the input-tax and output-tax system accounts.
Cash Flow	Posted journals involving bank ledger accounts. Exclude internal bank transfers; classify counterpart accounts. Supplier settlements are allocated proportionally across original bill lines/tax.

The cash-flow opening plus net movement equals closing bank balance. Account classifications influence presentation. Recoverable purchase tax is classified as operating cash flow by the implementation. These are operational reports; jurisdiction-specific statutory statements and tax filings require additional configuration/design.

Code: DashboardController; FinancialReports; OperationalReports; ReportData; ReportController; resources/js/Pages/Reports.

Attachments, exports, queues and audit

Private attachments

`attachments` stores company_id, model_type, model_id, disk, path, filename, MIME type and size. The binary is stored on the configured private filesystem, not inside the row. model_type is invoice, bill or expense; model_id points to that corresponding row. This polymorphic reference is validated by application logic rather than a generic SQL foreign key to three different tables.

Uploads support PDF/JPG/PNG/WebP up to 10 MB. Downloads check active tenant ownership and record permission. Paths are under company-specific attachment folders. Local storage is used now; private S3-compatible storage is available through configuration.

Report and business exports

Contact/invoice/bill XLSX exports and most PDF/report exports are generated synchronously from existing data. General-ledger/account-transactions PDF or XLSX requests create `report_exports` and enqueue `GenerateReportExport`. The record stores requesting user, company, report, format, filters, disk, path and queued/processing/ready/failed state. Generated binaries live under the company's private exports folder. Downloads recheck ownership and report permission. Spreadsheet cells are treated safely as strings to prevent formula injection.

Queue infrastructure

The local database queue stores pending work in `jobs`. A worker handles emails and exports. Completed queue rows are removed; `failed_jobs` tracks failures, and `job_batches` is available for Laravel batching. Local database cache/locks use `cache` and `cache_locks`. Redis/Horizon is prepared for Linux deployment; Horizon metadata is held in Redis rather than tenant accounting tables.

Audit trail

`audit_logs` records company, optional actor user, event, model class, record ID, JSON old/new values and timestamp. It records important financial, membership, permission and reconciliation actions. Some background actions have a null actor when no authenticated user is set; journal posting can pass an explicit actor. The log is not a complete version snapshot of every database change. Eloquent and PostgreSQL protect it against modification; access to its screen requires management permission.

Taxes, currencies and period locks

Setting	Database behavior
Company profile	Update companies.name/legal_name/email/phone/timezone and lock date. The current settings form does not populate company_settings.
Extra setting storage	company_settings offers company + setting_key + setting_value for extensions; a general key/value editor is not exposed.
Tax rates	tax_rates stores name, rate, type, default/active flags. Invoice/bill items copy the rate and calculated tax so later definition changes do not silently recalculate posted lines.
Currencies	currencies is shared seeded reference data. companies.currency, document currency and bank currency identify the base currency used by workflows.
Financial years	financial_years stores company-owned start/end dates and is_closed. Overlapping years are rejected.
Lock date	companies.locked_through blocks posting on/before that date. Lock dates can advance through the UI but cannot move backward.
Year closure	Set is_closed and advance locked_through. Future years cannot be closed. No separate closing journal is generated.

Each financial operation validates the posting period. A reversal is a new dated posting, so a closed period cannot be silently altered. Balance-sheet retained earnings are computed from cumulative revenue/expense journals; they are not a separately stored retained-earnings table.

Foreign-currency settlement and translation are disabled. `exchange_rates` exists for future rates, but current services require base-currency banks/documents. `products` and `payment_methods` likewise exist as reference schema without a full connected UI/workflow. Their existence does not mean inventory, product lookup or method selection is implemented.

Code: SettingsController; SettingsRequest; JournalService period checks; FoundationSeeder; migrations; Product/ExchangeRate/PaymentMethod models.

Subscriptions and platform plans

subscription_plans → subscriptions → subscription_invoices → subscription_payments

SaaS billing describes what a company pays to use Ledger. These records are separate from the company's customer invoices and supplier purchases.

Plans and company subscriptions

`subscription_plans` holds name, description, price, currency, monthly/yearly interval, JSON features/limits and availability. Platform administrators manage plans at `/admin/plans`.

`subscriptions` links company_id to plan_id with status, start/end date, trial_ends_at, cancelled_at and price/currency snapshot. A company's first subscription starts a 14-day trial. Selecting the same current plan returns it; changing plan cancels the prior record and creates a new full-price period. Proration is not implemented. Plan changes are blocked while the current subscription is past due.

Billing invoices and renewal

`subscription_invoices` stores subscription_id, number, amount, currency, due_date, period_start/end and open/paid status. Renewal uses a unique period-start key to prevent duplicate billing rows. A scheduled command processes due subscriptions under company context. Open overdue invoices mark the subscription past_due and block creating new restricted records. Invoice/member limits are enforced on relevant backend operations.

Verified platform payment

A platform administrator records a verified provider/reference value. The service inserts `subscription_payments`, marks the subscription invoice paid and reactivates a past_due subscription. It does not reactivate a cancelled plan. Reusing a reference for the same invoice returns the prior payment; using it for a different invoice is rejected.

> No tenant journal is posted for SaaS billing. An external card gateway, automatic recurring charge and provider webhook verification are not implemented. The payment action records a payment already verified by the platform administrator.

Code: SubscriptionService; SubscriptionController; PlanController; routes/console.php; Settings/Subscription.vue; Settings/Plans.vue.

Follow a purchase through the tables

These IDs and amounts are illustrative. The example uses company 10, supplier 20, bank 30, bill 40 and a 10% tax rate.

Step	Rows written and outcome
Supplier created	suppliers row 20 has company_id 10. No journal.
Bill draft saved	bills row 40 references supplier 20, subtotal 1,000, tax 100, total 1,100, draft status. bill_items references bill 40 and the purchase account. document_sequences advances.
Bill approved	journal_entries row 50 has source_type bill and source_id 40. Three journal_lines debit purchase 1,000 and input tax 100, credit payables 1,100. bills.journal_entry_id becomes 50; status approved.
Payment 400	journal_entries row 51 debits payables 400 and credits bank 400. bill_payments references bill 40, bank 30 and journal 51. bills.paid_total becomes 400; status partially_paid; outstanding 700.
Receipt attachment	attachments model_type bill/model_id 40; file binary stored privately.
Bank CSV imported	bank_transactions row 60 contains bank 30 and amount −400. No new purchase journal.
Bank row matched	reconciliations points row 60 to journal 51 and its bank-credit line. No duplicate payment.
Reports opened	Purchase expense 1,000, input tax asset 100, remaining payable 700 and bank movement −400 are derived from the journals. Bills/allocations supply the supplier aging details.

Which rows would a second payment create?

Paying the remaining 700 creates another bill_payments row and another settlement journal. It does not create another bill or another purchase-cost entry. Total paid becomes 1,100, outstanding becomes zero and bill status becomes paid.

A sale follows the mirror workflow

Customer → invoice/items → approval debits receivable and credits revenue/output tax → payment debits bank and credits receivable → statement match. A credit reduces original revenue/tax/receivable; a refund clears an existing customer credit using the bank.

Inspecting and running the project

Where to inspect local data

Use a SQLite database viewer to open `database/database.sqlite`. Start with the document header, follow its item rows, then its journal_entry_id and journal_lines. Always filter by company_id: IDs alone are not a company boundary. The examples below are read-only SQL queries; replace 10 and 40 with your selected company and bill.

```
SELECT * FROM bills WHERE company_id = 10 AND id = 40;

SELECT * FROM bill_items WHERE company_id = 10 AND bill_id = 40;

SELECT * FROM bill_payments WHERE company_id = 10 AND bill_id = 40;

SELECT * FROM journal_lines WHERE company_id = 10
AND journal_entry_id = (SELECT journal_entry_id FROM bills
WHERE company_id = 10 AND id = 40);
```

For a reversal, inspect journal_entries where reversal_of_id equals the original journal ID. For an imported bank match, inspect reconciliations by bank_transaction_id. For report totals, only posted journal entries qualify.

Local start commands

From the project folder in Git Bash:

```
/c/xampp1/php/php.exe artisan serve --host=127.0.0.1 --port=8087

node "C:/Program Files/nodejs/node_modules/npm/bin/npm-cli.js" run dev

/c/xampp1/php/php.exe artisan queue:work database --tries=1 --timeout=360 --memory=512
```

Run the server, Vite and queue worker as separate terminal commands; keep the queue options on the same line. Open `http://127.0.0.1:8087`. PHP in PowerShell uses `C:\xampp1\php\php.exe` instead. Already-built frontend assets can run without the Vite development server.

Seeders and tests

FoundationSeeder creates role defaults, permissions, currencies and plans. DatabaseSeeder runs FoundationSeeder. DemoSeeder adds development contacts and actual accounting transactions through services. It creates a random initial demo password once and does not reset an existing login. Routine user creation is available in the Users & roles screen.

`php artisan test` uses isolated in-memory SQLite. PostgreSQL has an optional dedicated test configuration for database-only safeguards. The last full SQLite run documented in the repository passed 105 tests; this PDF-generation task does not rerun application tests or alter transactions.

What exists and what remains

The project contains connected accounting workflows, not a static dashboard. At the same time, the original production-oriented brief is broader than the currently delivered functionality. This guide distinguishes those boundaries.

Implemented	Additional work
Company isolation, auth, user creation, role assignment and per-company role permission overrides.	Additional named custom role templates, notification inbox and automatic invitation email delivery.
Customers/suppliers, sales/bills, receipts/payments, credits/refunds and paid expenses.	Product catalog/inventory, supplier credits, multi-bill payment batches, expense adjustment/approval and percentage discounts.
Balanced journals, immutable posted records, reversals, locks and manual opening balances.	Separate manual-journal draft approval and explicit year-end closing journals.
Bank CSV import, one-to-one matching and statement closing checks.	Split/group matching, feeds, reconciliation undo and independently allocated session matches.
Posted-ledger reports, PDFs/XLSX, queued ledger exports and private attachments.	Statutory/local tax formats, further large-report streaming and load/concurrency review.
Plans, trials, subscription history, renewal invoice creation and verified manual payment records.	External gateway checkout, auto-collection, webhooks and proration.
SQLite local use and PostgreSQL schema support; Docker/Horizon/S3 configuration.	Foreign-currency accounting and runtime verification of deployment infrastructure, backups, SMTP and remote objects.

Storage concepts worth remembering

- User accounts live in users; ledger accounts live in accounts.
- Customer money receipts live in payments; supplier money payments live in bill_payments.
- Tenant sales invoices live in invoices; SaaS platform charges live in subscription_invoices.
- Business document details live in item tables; official debit/credit movements live in journal_lines.
- Imported bank records do not automatically create accounting postings.
- Files live on private storage; attachments/report_exports store metadata and paths.
- Most balances, dashboard cards and reports are calculated rather than stored in separate total tables.

Further project references

`docs/architecture.md`: ERD interpretation and accounting decisions.

`docs/progress.md`: implemented phases and verified boundaries.

`docs/deployment.md`: SQLite setup, PostgreSQL connection and deployment.

`docs/user-management.md`: creating users and changing access.

`docs/schema.md`: generated Mermaid schema diagram.

`tools/build-project-guide.php`: regenerate this PDF and its schema appendix.

APPENDIX A

Complete database reference

Generated from the migrated local database. These are schema definitions, not customer records. Each table below explains its purpose, columns, nullable fields, keys and relationships.

The local database uses SQLite. The migrations define the PostgreSQL deployment schema; SQLite type affinity and database safeguards differ. Financial precision and quantities are described in the main guide.

> 51 tables are present in this schema snapshot. All are covered in this appendix.

accounts

Chart of accounts: code, account type, parent, system purpose and cash-flow class.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
code	varchar	No
name	varchar	No
type	varchar	No
parent_id	integer	Yes
is_active	tinyint(1)	No
system_key	varchar	Yes
cash_flow_class	varchar	No

Unique: company_id + code.

Unique: company_id + id.

Unique: company_id + system_key.

Primary key: id.

FK: company_id + parent_id → accounts (company_id + id).

FK: company_id → companies (id).

attachments

Private file metadata and the invoice/bill/expense record to which the file belongs.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
model_type	varchar	No
model_id	integer	No
disk	varchar	No
file_path	varchar	No
file_name	varchar	No
mime_type	varchar	No
size	integer	No

Unique: company_id + id.

Primary key: id.

FK: company_id → companies (id).

audit_logs

Append-only actor/company/event history with affected record and old/new values.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
user_id	integer	Yes
event	varchar	No
model_type	varchar	No
model_id	integer	No
old_values	text	Yes
new_values	text	Yes
created_at	datetime	No

Primary key: id.

FK: user_id → users (id).

FK: company_id → companies (id).

bank_accounts

Company bank metadata and its associated asset ledger account.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
name	varchar	No
bank_name	varchar	Yes
account_number	varchar	Yes
currency	varchar	No
account_id	integer	No
linked_status	varchar	No

Unique: company_id + account_id.

Unique: company_id + id.

Primary key: id.

FK: company_id + account_id → accounts (company_id + id).

FK: company_id → companies (id).

bank_transactions

Imported signed bank-statement rows and duplicate-detection fingerprints.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
bank_account_id	integer	No
date	date	No
description	text	No
amount	numeric	No
transaction_type	varchar	No
reference	varchar	Yes
import_hash	varchar	No
imported_at	datetime	No

Unique: company_id + bank_account_id + import_hash.

Unique: company_id + id.

Primary key: id.

FK: company_id + bank_account_id → bank_accounts (company_id + id).

FK: company_id → companies (id).

bill_items

Purchase lines linked to expense/asset accounts, with quantity, discount and tax.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
bill_id	integer	No
account_id	integer	No
tax_rate_id	integer	Yes
description	text	No
quantity	numeric	No
unit_price	numeric	No
discount	numeric	No
tax_rate	numeric	No
subtotal	numeric	No
tax_amount	numeric	No
total	numeric	No

Unique: company_id + id.

Primary key: id.

FK: company_id + tax_rate_id → tax_rates (company_id + id).

FK: company_id + account_id → accounts (company_id + id).

FK: company_id + bill_id → bills (company_id + id).

FK: company_id → companies (id).

bill_payments

Supplier payments allocated to one bill, linked to a bank and journal.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
bill_id	integer	No
bank_account_id	integer	No
journal_entry_id	integer	No
payment_method	varchar	No
amount	numeric	No
paid_at	date	No
reference	varchar	Yes
notes	text	Yes
idempotency_key	varchar	No
request_hash	varchar	No

Unique: company_id + id.

Unique: company_id + idempotency_key.

Primary key: id.

FK: company_id + journal_entry_id → journal_entries (company_id + id).

FK: company_id + bank_account_id → bank_accounts (company_id + id).

FK: company_id + bill_id → bills (company_id + id).

FK: company_id → companies (id).

bills

Supplier purchase document header, totals, payment summary and posted-journal link.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
supplier_id	integer	No
bill_number	varchar	No
bill_date	date	No
due_date	date	No
currency	varchar	No
status	varchar	No
subtotal	numeric	No
tax_total	numeric	No
total	numeric	No
paid_total	numeric	No
credited_total	numeric	No
notes	text	Yes
journal_entry_id	integer	Yes

Unique: company_id + bill_number.

Unique: company_id + id.

Primary key: id.

FK: company_id + journal_entry_id → journal_entries (company_id + id).

FK: company_id + supplier_id → suppliers (company_id + id).

FK: company_id → companies (id).

cache

Database-backed cache values used by the current local configuration.

Column	Database type	Nullable
key	varchar	No
value	text	No
expiration	integer	No

Primary key: key.

cache_locks

Database-backed distributed locks, including invoice-email delivery locks.

Column	Database type	Nullable
key	varchar	No
owner	varchar	No
expiration	integer	No

Primary key: key.

companies

Company profile, base currency, timezone, status and accounting lock date.

Column	Database type	Nullable
id	integer	No
name	varchar	No
legal_name	varchar	Yes
email	varchar	Yes
phone	varchar	Yes
currency	varchar	No
timezone	varchar	No
financial_year_start_date	date	No
locked_through	date	Yes
status	varchar	No
created_at	datetime	Yes
updated_at	datetime	Yes

Primary key: id.

FK: currency → currencies (code).

company_invitations

Recipient, role, hashed invitation token, expiry and acceptance timestamp.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
email	varchar	No
token_hash	varchar	No
role_id	integer	No
expires_at	datetime	No
accepted_at	datetime	Yes

Unique: company_id + id.

Unique: token_hash.

Primary key: id.

FK: role_id → roles (id).

FK: company_id → companies (id).

company_permission_overrides

Per-company allowed/denied overrides for role permissions.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
role_id	integer	No
permission_id	integer	No
allowed	tinyint(1)	No
created_at	datetime	Yes
updated_at	datetime	Yes

Unique: company_id + role_id + permission_id.

Primary key: id.

FK: permission_id → permissions (id).

FK: role_id → roles (id).

FK: company_id → companies (id).

company_settings

Additional company key/value settings; the current profile form writes companies instead.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
setting_key	varchar	No
setting_value	text	Yes

Unique: company_id + id.

Unique: company_id + setting_key.

Primary key: id.

FK: company_id → companies (id).

company_users

Membership linking a user to a company, with its role and active-access flag.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
user_id	integer	No
role_id	integer	No
is_active	tinyint(1)	No

Unique: company_id + id.

Unique: company_id + user_id.

Primary key: id.

FK: role_id → roles (id).

FK: user_id → users (id).

FK: company_id → companies (id).

credit_note_items

Credited original invoice lines, before-tax amounts and tax adjustments.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
credit_note_id	integer	No
account_id	integer	No
description	text	No
amount	numeric	No
tax_amount	numeric	No
invoice_item_id	integer	No

Unique: company_id + id.

Primary key: id.

FK: company_id + invoice_item_id → invoice_items (company_id + id).

FK: company_id → companies (id).

FK: company_id + credit_note_id → credit_notes (company_id + id).

FK: company_id + account_id → accounts (company_id + id).

credit_notes

Posted customer credit header linked to the original invoice.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
invoice_id	integer	No
credit_number	varchar	No
date	date	No
amount	numeric	No
reason	text	No
status	varchar	No
journal_entry_id	integer	Yes

Unique: company_id + credit_number.

Unique: company_id + id.

Primary key: id.

FK: company_id + journal_entry_id → journal_entries (company_id + id).

FK: company_id + invoice_id → invoices (company_id + id).

FK: company_id → companies (id).

currencies

Shared supported currency codes, names, symbols and decimal-place reference data.

Column	Database type	Nullable
id	integer	No
code	varchar	No
name	varchar	No
symbol	varchar	No
decimal_places	integer	No
is_active	tinyint(1)	No
created_at	datetime	Yes
updated_at	datetime	Yes

Unique: code.

Primary key: id.

customer_refunds

Cash paid back against a credit note, with bank and journal links.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
credit_note_id	integer	No
bank_account_id	integer	No
journal_entry_id	integer	No
amount	numeric	No
date	date	No
idempotency_key	varchar	No
request_hash	varchar	No

Unique: company_id + id.

Unique: idempotency_key.

Primary key: id.

FK: company_id + journal_entry_id → journal_entries (company_id + id).

FK: company_id + bank_account_id → bank_accounts (company_id + id).

FK: company_id + credit_note_id → credit_notes (company_id + id).

FK: company_id → companies (id).

customers

Company customer contact information and active/archived status.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
name	varchar	No
email	varchar	Yes
phone	varchar	Yes
address	text	Yes
tax_number	varchar	Yes
status	varchar	No

Unique: company_id + id.

Primary key: id.

FK: company_id → companies (id).

document_sequences

Next per-company invoice, bill and credit-note number.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
type	varchar	No
next_number	integer	No

Unique: company_id + id.

Unique: company_id + type.

Primary key: id.

FK: company_id → companies (id).

exchange_rates

Shared dated currency-rate reference schema; foreign-currency posting is not enabled.

Column	Database type	Nullable
id	integer	No
base_currency	varchar	No
quote_currency	varchar	No
date	date	No
rate	numeric	No
created_at	datetime	Yes
updated_at	datetime	Yes

Unique: base_currency + quote_currency + date.

Primary key: id.

expense_categories

Expense category tree and the expense account assigned to each category.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
name	varchar	No
parent_id	integer	Yes
account_id	integer	No

Unique: company_id + id.

Primary key: id.

FK: company_id + account_id → accounts (company_id + id).

FK: company_id + parent_id → expense_categories (company_id + id).

FK: company_id → companies (id).

expenses

Direct paid expenses, category, bank, tax, posting status and retry protection.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
category_id	integer	No
bank_account_id	integer	No
journal_entry_id	integer	Yes
tax_rate_id	integer	Yes
date	date	No
description	text	No
amount	numeric	No
tax_amount	numeric	No
payment_method	varchar	No
status	varchar	No
idempotency_key	varchar	Yes
request_hash	varchar	Yes

Unique: company_id + id.

Unique: company_id + idempotency_key.

Primary key: id.

FK: company_id + tax_rate_id → tax_rates (company_id + id).

FK: company_id + journal_entry_id → journal_entries (company_id + id).

FK: company_id + bank_account_id → bank_accounts (company_id + id).

FK: company_id + category_id → expense_categories (company_id + id).

FK: company_id → companies (id).

failed_jobs

Failed queued-job metadata used for operational troubleshooting.

Column	Database type	Nullable
id	integer	No
uuid	varchar	No
connection	text	No
queue	text	No
payload	text	No
exception	text	No
failed_at	datetime	No

Unique: uuid.

Primary key: id.

financial_years

Company accounting-year dates and closed-period status.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
name	varchar	No
start_date	date	No
end_date	date	No
is_closed	tinyint(1)	No

Unique: company_id + id.

Unique: company_id + start_date.

Primary key: id.

FK: company_id → companies (id).

invoice_items

Invoice lines, account, quantity, unit price, discount and tax snapshot.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
invoice_id	integer	No
account_id	integer	No
tax_rate_id	integer	Yes
description	text	No
quantity	numeric	No
unit_price	numeric	No
discount	numeric	No
tax_rate	numeric	No
subtotal	numeric	No
tax_amount	numeric	No
total	numeric	No

Unique: company_id + id.

Primary key: id.

FK: company_id + tax_rate_id → tax_rates (company_id + id).

FK: company_id + account_id → accounts (company_id + id).

FK: company_id + invoice_id → invoices (company_id + id).

FK: company_id → companies (id).

invoices

Sales document header, customer, dates, totals, status and posted-journal link.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
customer_id	integer	No
invoice_number	varchar	No
issue_date	date	No
due_date	date	No
currency	varchar	No
status	varchar	No
subtotal	numeric	No
tax_total	numeric	No
total	numeric	No
paid_total	numeric	No
credited_total	numeric	No
notes	text	Yes
journal_entry_id	integer	Yes

Unique: company_id + id.

Unique: company_id + invoice_number.

Primary key: id.

FK: company_id + journal_entry_id → journal_entries (company_id + id).

FK: company_id + customer_id → customers (company_id + id).

FK: company_id → companies (id).

job_batches

Laravel batch-job bookkeeping; no dedicated batch workflow is currently exposed.

Column	Database type	Nullable
id	varchar	No
name	varchar	No
total_jobs	integer	No
pending_jobs	integer	No
failed_jobs	integer	No
failed_job_ids	text	No
options	text	Yes
cancelled_at	integer	Yes
created_at	integer	No
finished_at	integer	Yes

Primary key: id.

jobs

Pending database queue payloads for invoice emails and background exports.

Column	Database type	Nullable
id	integer	No
queue	varchar	No
payload	text	No
attempts	integer	No
reserved_at	integer	Yes
available_at	integer	No
created_at	integer	No

Primary key: id.

journal_entries

Accounting entry header: date, reference, source, author, posting and reversal link.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
date	date	No
reference	varchar	No
description	text	Yes
source_type	varchar	Yes
source_id	integer	Yes
status	varchar	No
created_by	integer	No
posted_at	datetime	Yes
reversal_of_id	integer	Yes

Unique: company_id + id.

Unique: company_id + reversal_of_id.

Unique: company_id + source_type + source_id.

Primary key: id.

FK: company_id + reversal_of_id → journal_entries (company_id + id).

FK: created_by → users (id).

FK: company_id → companies (id).

journal_lines

The debit/credit amounts posted to individual ledger accounts.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
journal_entry_id	integer	No
account_id	integer	No
debit	numeric	No
credit	numeric	No
description	text	Yes

Unique: company_id + id.

Primary key: id.

FK: company_id + account_id → accounts (company_id + id).

FK: company_id + journal_entry_id → journal_entries (company_id + id).

FK: company_id → companies (id).

migrations

Which schema migrations have run and their batch numbers.

Column	Database type	Nullable
id	integer	No
migration	varchar	No
batch	integer	No

Primary key: id.

password_reset_tokens

Hashed password-recovery tokens keyed by login email.

Column	Database type	Nullable
email	varchar	No
token	varchar	No
created_at	datetime	Yes

Primary key: email.

payment_methods

Company payment-method reference schema; current workflows use bank payments.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
name	varchar	No
type	varchar	No
details	text	Yes
is_active	tinyint(1)	No

Unique: company_id + id.

Primary key: id.

FK: company_id → companies (id).

payments

Customer receipts allocated to an invoice, linked to a bank and journal.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
invoice_id	integer	No
bank_account_id	integer	No
journal_entry_id	integer	No
payment_method	varchar	No
amount	numeric	No
paid_at	date	No
reference	varchar	Yes
notes	text	Yes
idempotency_key	varchar	No
request_hash	varchar	No

Unique: company_id + id.

Unique: company_id + idempotency_key.

Primary key: id.

FK: company_id + journal_entry_id → journal_entries (company_id + id).

FK: company_id + bank_account_id → bank_accounts (company_id + id).

FK: company_id + invoice_id → invoices (company_id + id).

FK: company_id → companies (id).

permissions

Shared permission names checked by protected backend operations.

Column	Database type	Nullable
id	integer	No
name	varchar	No

Unique: name.

Primary key: id.

products

Product/service reference schema; a catalog workflow is not currently connected.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
code	varchar	No
name	varchar	No
description	text	Yes
unit_price	numeric	No
account_id	integer	No
is_active	tinyint(1)	No

Primary key: id.

Unique: company_id + code.

Unique: company_id + id.

FK: company_id + account_id → accounts (company_id + id).

FK: company_id → companies (id).

reconciliation_sessions

Statement end date, closing balance and open/closed reconciliation certificate.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
bank_account_id	integer	No
statement_date	date	No
closing_balance	numeric	No
status	varchar	No
closed_at	datetime	Yes

Primary key: id.

Unique: company_id + id.

FK: company_id + bank_account_id → bank_accounts (company_id + id).

FK: company_id → companies (id).

reconciliations

One-to-one matching of a bank-statement transaction to a posted bank journal line.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
bank_transaction_id	integer	No
journal_entry_id	integer	No
journal_line_id	integer	No
session_id	integer	Yes
matched_amount	numeric	No
status	varchar	No
reconciled_at	datetime	No

Primary key: id.

Unique: company_id + bank_transaction_id.

Unique: company_id + id.

Unique: company_id + journal_line_id.

FK: company_id + session_id → reconciliation_sessions (company_id + id).

FK: company_id + journal_line_id → journal_lines (company_id + id).

FK: company_id + journal_entry_id → journal_entries (company_id + id).

FK: company_id + bank_transaction_id → bank_transactions (company_id + id).

FK: company_id → companies (id).

report_exports

Owner, company, filters, format, job status and private generated-report path.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
user_id	integer	No
report	varchar	No
format	varchar	No
filters	text	No
status	varchar	No
disk	varchar	No
path	varchar	Yes
error	text	Yes
created_at	datetime	Yes
updated_at	datetime	Yes

Primary key: id.

FK: user_id → users (id).

FK: company_id → companies (id).

role_permissions

Default permission assignments for each predefined role.

Column	Database type	Nullable
role_id	integer	No
permission_id	integer	No

Primary key: role_id + permission_id.

FK: permission_id → permissions (id).

FK: role_id → roles (id).

roles

Shared predefined company role names, such as Owner and Accountant.

Column	Database type	Nullable
id	integer	No
name	varchar	No
guard_name	varchar	No
created_at	datetime	Yes
updated_at	datetime	Yes

Primary key: id.

Unique: name.

sessions

Database-backed authentication/session payloads, including active-company selection.

Column	Database type	Nullable
id	varchar	No
user_id	integer	Yes
ip_address	varchar	Yes
user_agent	text	Yes
payload	text	No
last_activity	integer	No

Primary key: id.

subscription_invoices

Platform subscription charges for unique billing periods, separate from sales invoices.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
subscription_id	integer	No
number	varchar	No
amount	numeric	No
currency	varchar	No
due_date	date	No
status	varchar	No
period_start	date	No
period_end	date	No

Primary key: id.

Unique: company_id + id.

Unique: company_id + subscription_id + period_start.

Unique: number.

FK: company_id + subscription_id → subscriptions (company_id + id).

FK: company_id → companies (id).

subscription_payments

Platform-admin-verified settlement of subscription invoices.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
subscription_invoice_id	integer	No
amount	numeric	No
provider_reference	varchar	No
paid_at	datetime	No

Primary key: id.

Unique: company_id + id.

Unique: provider_reference.

FK: company_id + subscription_invoice_id → subscription_invoices (company_id + id).

FK: company_id → companies (id).

subscription_plans

Platform plan prices, interval, currency, features and active availability.

Column	Database type	Nullable
id	integer	No
name	varchar	No
description	text	Yes
price	numeric	No
currency	varchar	No
interval	varchar	No
features	text	No
is_active	tinyint(1)	No
created_at	datetime	Yes
updated_at	datetime	Yes

Primary key: id.

subscriptions

Company subscription history, selected plan, trial, cancellation and billing-period dates.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
plan_id	integer	No
status	varchar	No
start_date	date	No
end_date	date	Yes
trial_ends_at	datetime	Yes
cancelled_at	datetime	Yes
amount	numeric	No
currency	varchar	No

Primary key: id.

Unique: company_id + id.

FK: plan_id → subscription_plans (id).

FK: company_id → companies (id).

suppliers

Company supplier contact information and active/archived status.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
name	varchar	No
email	varchar	Yes
phone	varchar	Yes
address	text	Yes
tax_number	varchar	Yes
status	varchar	No

Primary key: id.

Unique: company_id + id.

FK: company_id → companies (id).

tax_rates

Company tax definitions used to snapshot tax on document lines and expenses.

Column	Database type	Nullable
id	integer	No
company_id	integer	No
created_at	datetime	Yes
updated_at	datetime	Yes
name	varchar	No
rate	numeric	No
type	varchar	No
is_default	tinyint(1)	No
is_active	tinyint(1)	No

Primary key: id.

Unique: company_id + id.

FK: company_id → companies (id).

users

Global login identity, password hash, email verification and platform-administrator flag.

Column	Database type	Nullable
id	integer	No
name	varchar	No
email	varchar	No
email_verified_at	datetime	Yes
password	varchar	No
remember_token	varchar	Yes
created_at	datetime	Yes
updated_at	datetime	Yes
is_platform_admin	tinyint(1)	No

Primary key: id.

Unique: email.